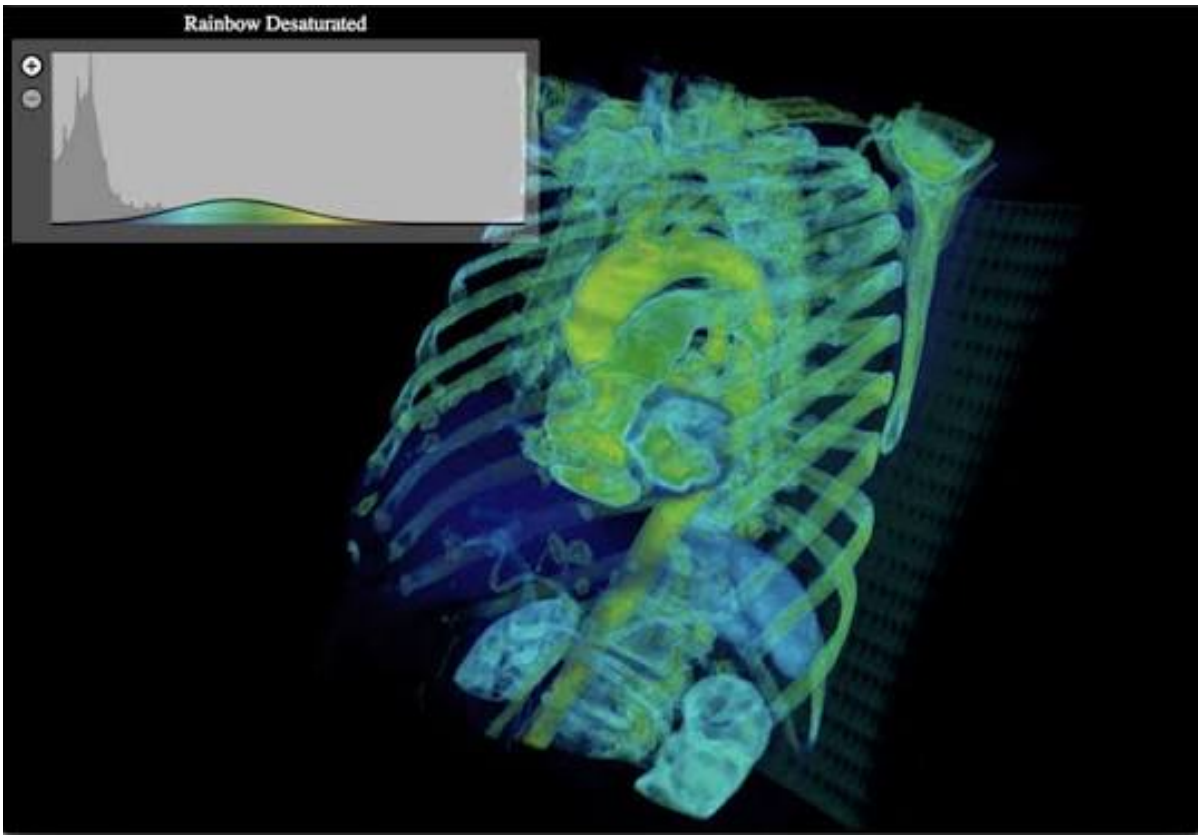
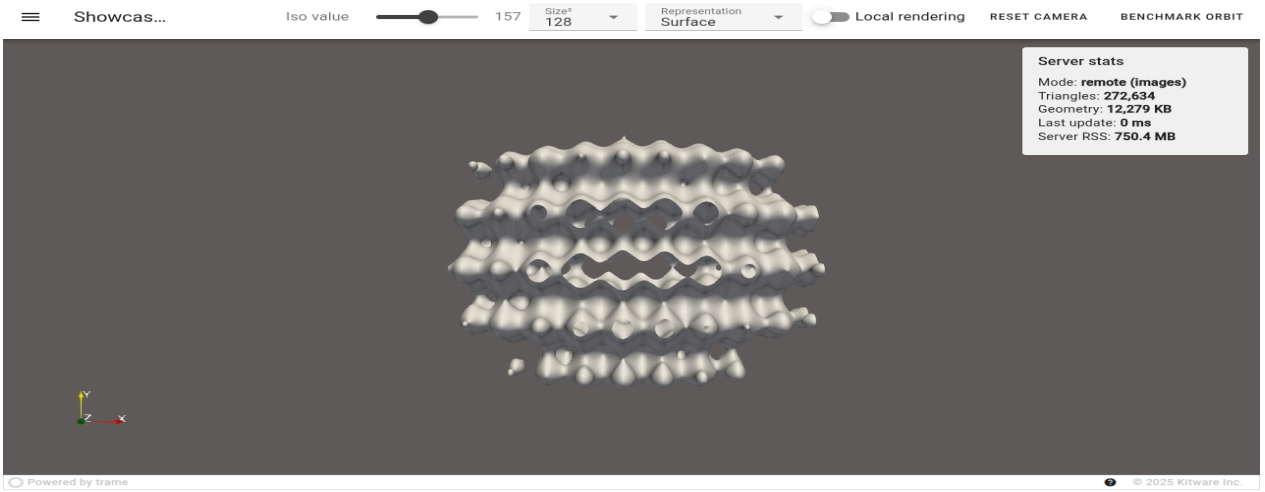
EVALUATION OF VISUALISATION SYSTEMS

AN REPORT ON

PARAVIEW WEB + VTK.JS

By Balasubramanian (2025121002)





0.TABLE OF CONTENTS

Sno	title	Page no
1.	Introduction and Scope	3
2.	Background and architecture	5
3.	Installation and reproducibility	6
4.	Showcase A	7
5.	Showcase B	11
6.	Feature comparison matrix	16
7.	Benchmark	18
8.	ISO/IEC 25010 evaluation	20
9.	Munzner nested-model analysis	22
10.	Advantages and disadvantage	24
11.	Legacy ParaViewWeb (desk note)	26
12.	Research gaps and future work	27
13.	Conclusion and recommendations	29
14.	Appendix	31

1.Introduction and scope

Paraview and VTK are 2 important tools used in visualization of scientific data.
To make these visualizers available easily and usable as part of web articles,paraview web and vtk.js are 2 frameworks which helps in rendering visualization in browsers without downloading anything. Vtk.js renders in browser itself whereas paraview web renders on a server and streams it to the brower

The showcases built are:

a) vtk.js for client-side rendering
b) ParaView 6.2 with trame for server-side and hybrid rendering

This was done because from paraview 5.13 onwards legacy web applications are no longer bundled with paraview

This evaluation contains a overview of the several features of vtk.js and what it can and cannot render as well as ParaView 6.2 with trame for server side and hybrid rendering.
It benchmarks the frame rate,time to first render ,memory usage and network traffic of the render too with all evaluation done on the chrome browser on popOs(debian based distribution of linux OS) with integrated GPU(Intel iris Xe) enabled for the browser.
The evaluations also scored these frameworks against the 8 quality characteristics of ISO 25010 for software quality.

This work mainly explores 4 things

1. What can vtk.js and ParaView with trame do for visualizing 3D scientific data, and which of these capabilities can be demonstrated in working applications?
2. How does rendering in the browser (client-side) compare with rendering on a server (server-side) in frame rate, time to first render, memory use and network traffic?
3. How easy is it to install and reproduce each system, and how sustainable is it, given the maintenance status of the underlying projects?
4. For which use cases is each approach suited, and where does it break down?

So in essence the contributions of this work are

1) two working showcase applications, a vtk.js volume viewer and a trame/ParaView application with a switch between client-side and server-side rendering, plus a topology-analysis app that uses the Topology ToolKit
2) a feature matrix of 75 capabilities comparing vtk.js, ParaView with trame and the legacy ParaViewWeb applications, marking each as demonstrated, documented or unsupported
3) a benchmark of local and remote rendering on a single laptop
4) a scoring of both frameworks against ISO/IEC 25010
5) a set of scripts that reproduce the setup and the results

The structure of the report is that in the next section i will discuss the architecture of both approaches followed by installation and reproducibility.then we will move on the main contribution of the 2 showcases and their features.The later sections will describe the benchmarking process and results and summarizes munzers nested model.The concluding parts will report the advantages and disadvantages and a small note on legacy paraview applications along with research gaps and future work

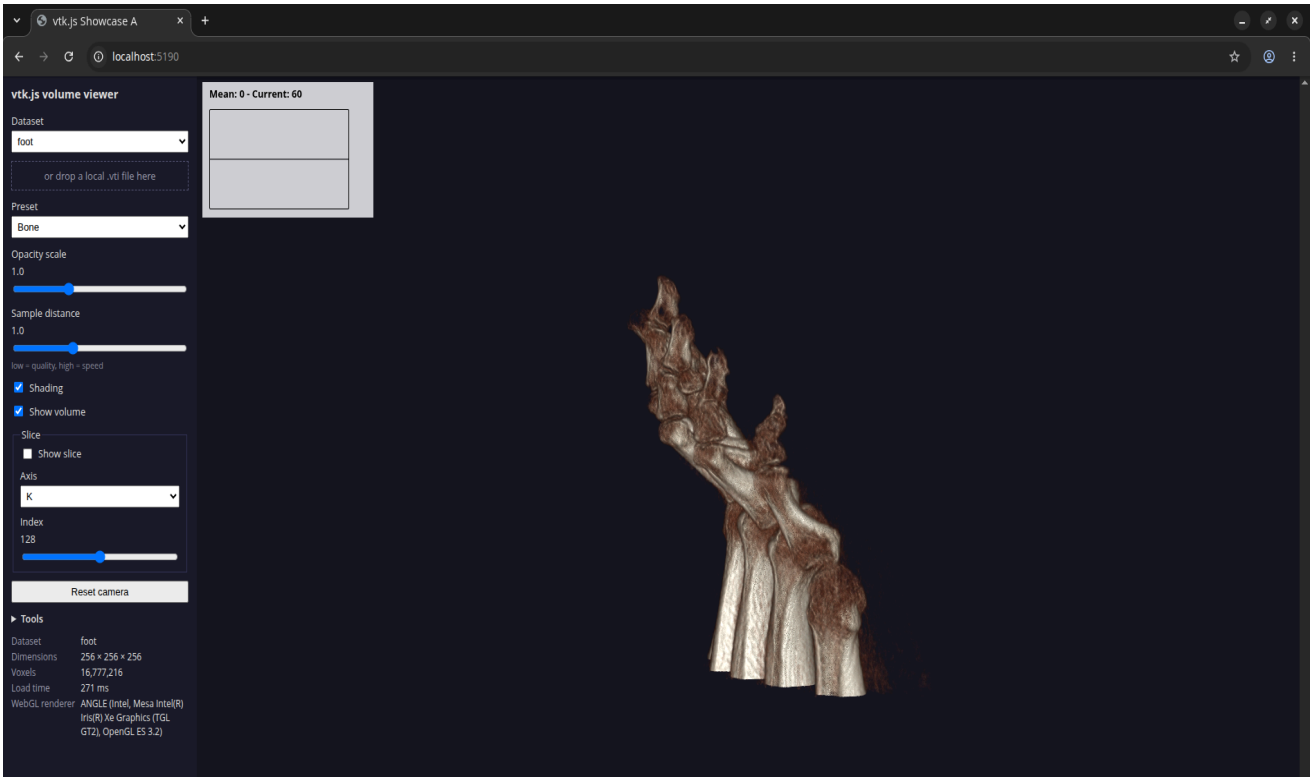


Fig1: showcase A vtk.js

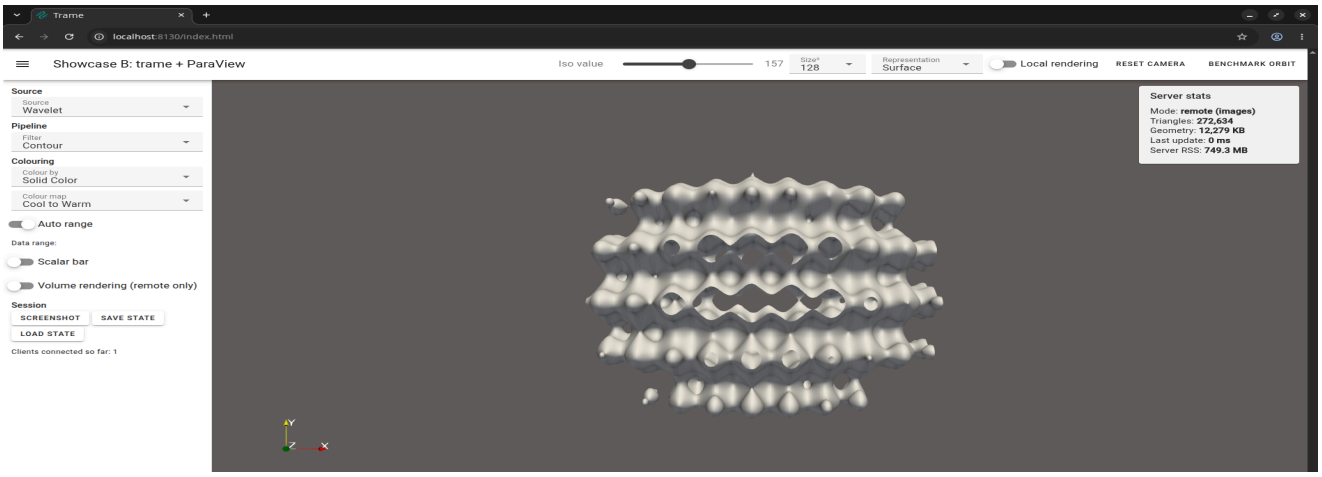


Fig2: Showcase B paraview+ trame

2.Background and Architecture

VTK is a C++ library which supports various volume rendering possibilities including isocontour visualization.Paraview is an application that is built on top of VTK.
What i implemented vtk.js is a javascript reimplementation of VTK rendering pipeline for the browser using WEBGL2.Similarly trame is Kitware's python framework for building web

apps. it uses wslink(Websockets) for communication and can load paraview or VTK on the server.

For showcase A,no server code runs and only static files are served.The browser then downloads .vti data and vtk.js draws it with WebGL2 using the inbuilt GPU.This interaction requires no network after the data has loaded from the server to the client.

For showcase B,a trame app runs under pvpython and Paraview builds the pipeline on the server.The are two modes, remote and local. In remote mode the server first renders, then compresses each frame as an image and streams it over a WebSocket (wslink).Mouse events recursively go back to the server. In the local mode in contrast,the server serialises the geometry only once, and vtk.js renders it in the browser. Switching between both modes triggers a full update

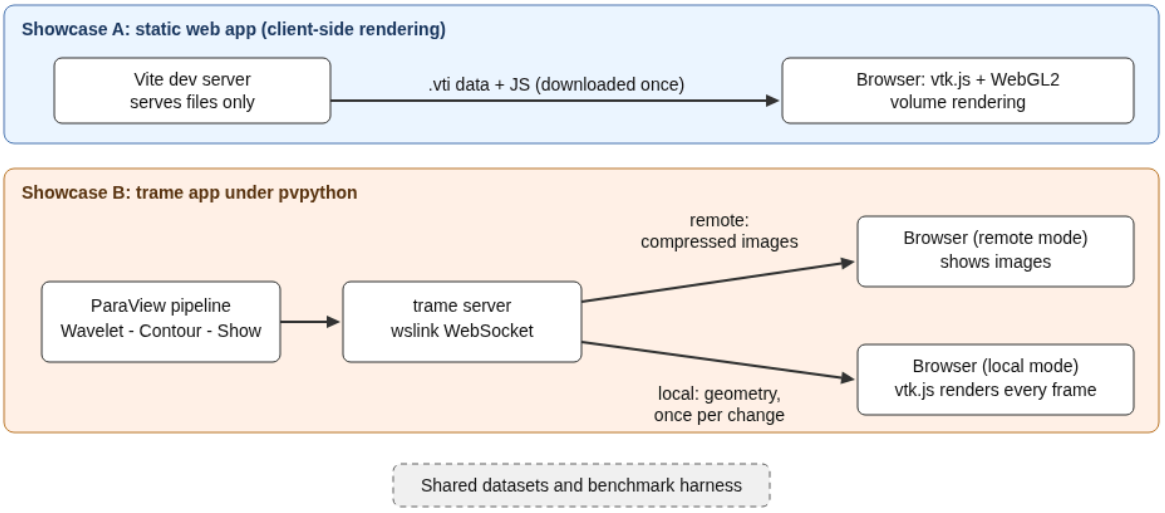


Fig3: the rendering architecture

3.Installation and Reproducibilty

Everything related to running the demos was installed step by step and each step is precisely given in install log in the repository with how long it took and what went wrong.

The setup is repeatable with scripts/setup.sh which will create the python environment,install packages,download and convert datasets.

Two separate python environments are created to separate Paraview + trame which never should contain a pip installed vtk because paraview ships its own copy and it may clash. The benchmarking tools have a second environment which has vtk.The respective versions used are vtk.js 37.4.6, Vite 8.3.5, ParaView 6.2.0, trame 4.0.0, trame-vtk 2.11.17, Node.js 24.13 and Python 3.12.

If you are trying to reproduce what i did here are certain issues i encountered:

Issue	Effect	Resolution
The dataset host (klacansky.com) refused HTTPS connections	Downloads failed	Switched to plain HTTP and recorded SHA-256 checksums to catch bad files
The same host was slow(5kb/s)	setup.sh looked frozen	resume partial files, give up on trickling connections, and retry
A dataset URL pointed to vtk.js's JSON directory format, not an XML .vti file	The smoke-test volume could not be read	Replace it with a wavelet volume generated locally
vtk.js in Vite showed a blank page	Showcase A did not render	Install the events package and aliase it in the Vite config

To test reproducibility the repo was newly cloned from github and the setup process was followed and showcases A and B rendered upon following instructions.

However the limitations are that i only tested on Linux and Chrome

4.Showcase A

Showcase A is a single-page web app built with Vite. vtk.js does all the rendering in the browser with WebGL2 and there is no server code at all.The server only hands out the files

I used four datasets(a foot, an aneurism and a bonsai tree, each at 256³ voxels, and a tooth at 103×94×161) from the Open SciVis collection and firstly converted them to .vti files.For the benchmark I also made smaller copies of the foot at 64³ and 128³

The base viewer does direct volume rendering with three transfer-function presets (Grayscale, Bone and Vessel), plus sliders for opacity and sample distance and a toggle for shading. It also has a single slice view along the I, J or K axis, drag-and-drop loading of your own .vti files, and an info panel with the dataset size, load time and the WebGL renderer in use. vtk.js's FPS monitor shows the mean and current frame rate on screen

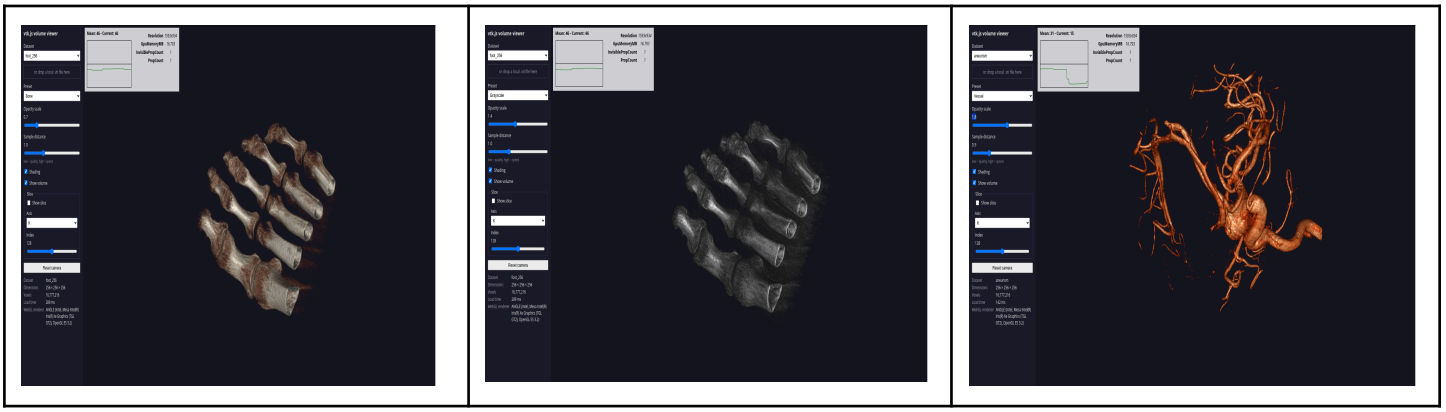


Fig2: The 3 transfer function presets Grayscale,bone and vessel respectively shown from left to right

Beyond the base viewer, I added a collapsible Tools panel to see how much of vtk.js I could use in practice. Table 1 lists what it contains and how well each part worked.

Tool	What it does	Notes
Isosurface	Marching cubes in the browser, with an iso-value slider	Foot at 128^3 gave about 133,000 triangles in 144 ms. It runs on the main thread, so a 256^3 volume may freeze the page
Orthogonal slices	Three planes (I, J, K) shown together	Planes can look black where they cut through empty regions
Probe	Click to read the voxel position and value	Uses a cell picker; works on the volume, slices,meshes and isosurface
Clipping	Six clipping planes on the volume and isosurface	used plane sliders instead of the image-cropping widget
Legend and axes	Scalar bar and orientation axes	Needed an extra vtk.js profile import to work
Transfer-function editor	Drag opacity points on	Edits opacity only, and switching preset resets it
Mesh loading	Loads OBJ meshes alongside the volume	VTP and STL reader code exists but I never tested it with real files
Measure and export	Distance line between two points, and PNG export	The distance is measured in a plane facing the camera, not on a surface
WebXR	An "Enter VR" button	Tested only in an emulator, with no real headset

Table 1

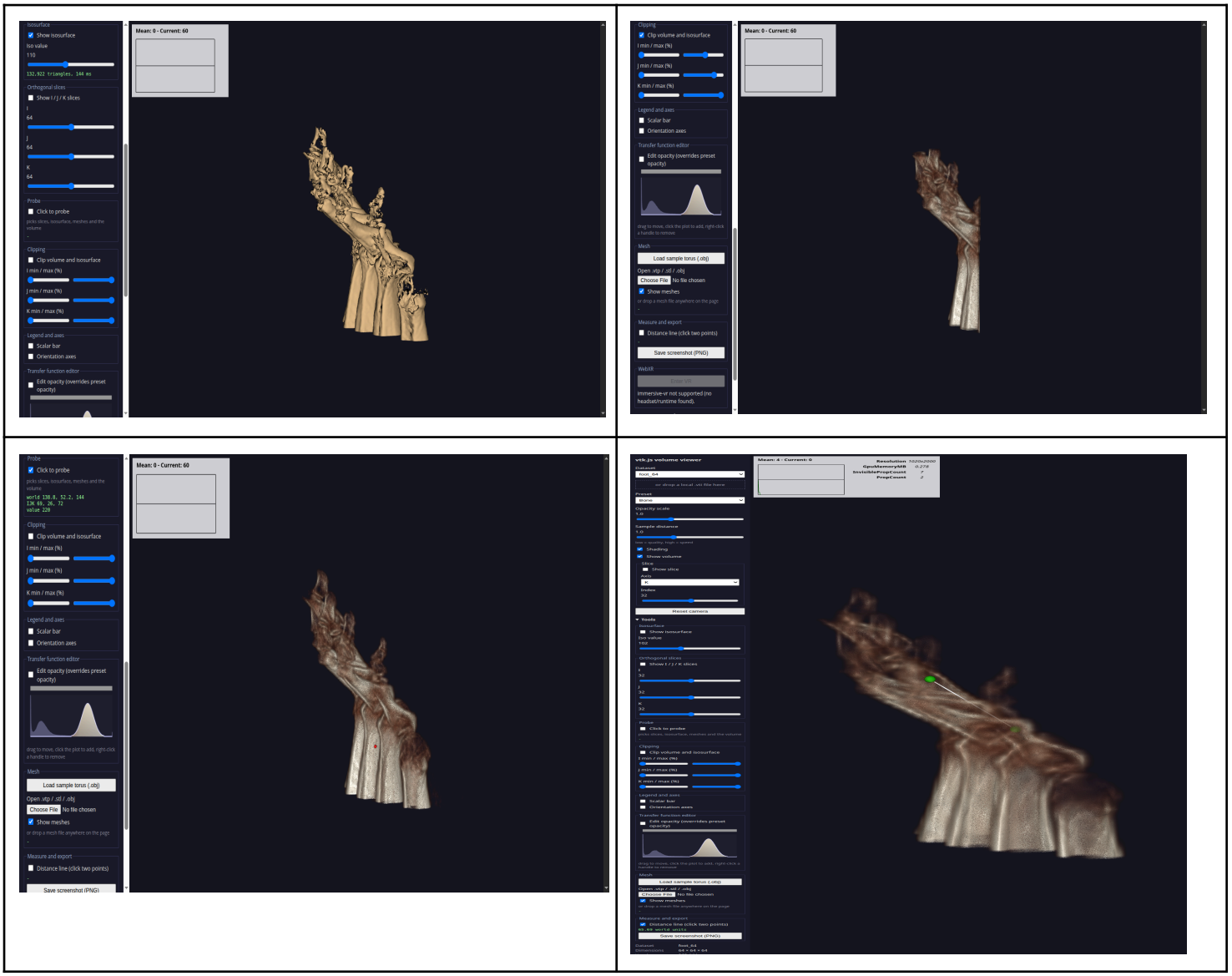


Fig3 : depicting a) isosurface b)clipped volume c)probe(notice the small red dot clicked) d)distance measure(green line)

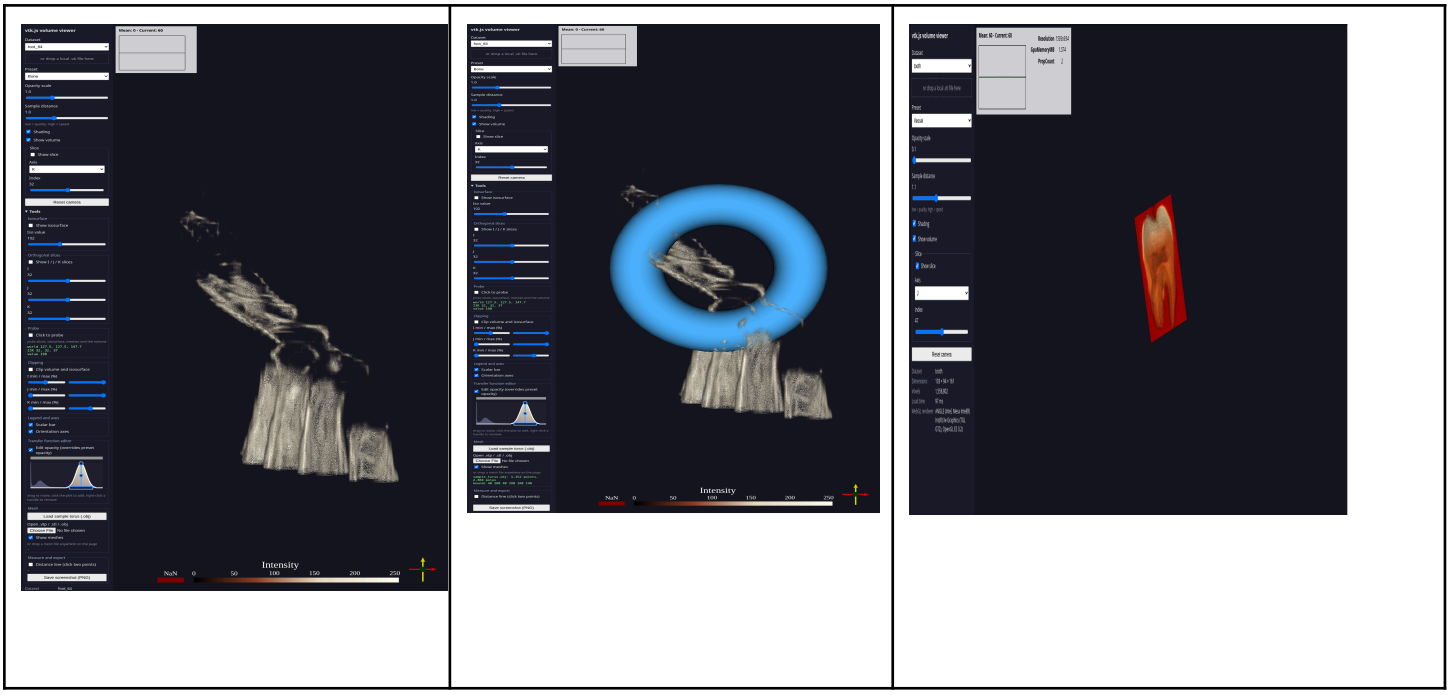


Fig4 : depicting a) custom transfer function b)an torus mesh imported c)slice of a tooth

Observations:

Rendering quality: The presets do show different things in the same data. On the foot, Grayscale gives a plain X-ray-like picture of the bones, Bone preset separates the bone (pale) from the soft tissue (brown), and Vessel used on the aneurism brings out the branching blood vessels clearly in orange

Loading and memory: Loading took 97 ms for the tooth and 134–289 ms for the 256³ volumes, because the files come from the local server. Memory use was small: the overlay reported about 17 MB of GPU memory for a 256³ volume and 1.6 MB for the tooth, and the browser's JavaScript memory in the benchmark was 20 to 41 MB. These times are for a local server, so a real network would add download time

Speed: The numbers come from two sources that I kept separate. The on-screen FPS overlay, read during interaction in a 1593×934 window, showed a mean of 39–46 fps on the 256³ foot and aneurism, and a steady 60 on the much smaller tooth. The scripted benchmark, in a smaller window, told the same story. the 64³ and 128³ foot held 60 fps, and the 256³ foot dropped to a median of 30 fps, with its slowest second at about 25 fps. So on this integrated GPU, a 256³ volume no longer reaches the display's 60 Hz limit, while volumes up to 128³ do.

5.Showcase B

Showcase B is a web app written in Python with trame and run with ParaView's pvpython. ParaView builds the visualization pipeline on the server, and the browser shows the result. The toolbar has :
1)iso-value slider
2)a size selector (64³, 128³ or 256³ wavelet volumes)
3)a Surface/Wireframe switch
4)Reset camera
5)"Local rendering" switch
6)Benchmark orbit button that spins the camera 180 steps of 2° and reports the time.

A Server stats card shows the current mode, triangle count, geometry size, time of the last update and the server's memory use. A version built on plain VTK, without ParaView, serves as a fallback, and a software-rendered check gave it identical triangle counts.

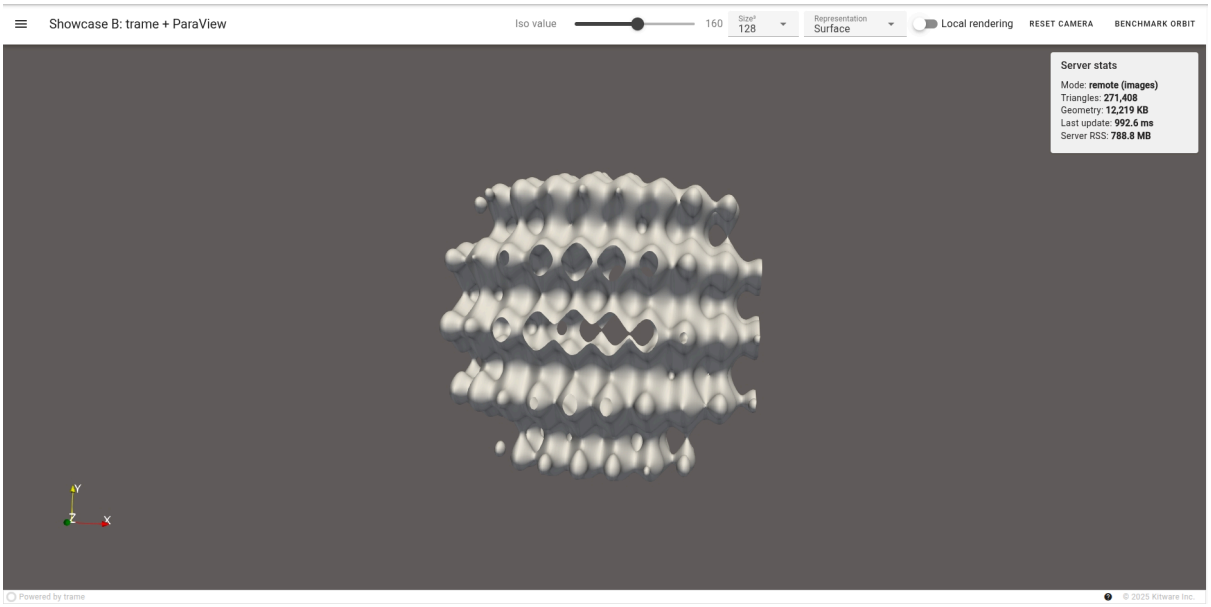


Fig5: remote rendering(see top right stats bar rendering MODE)

A Pipeline drawer was added to see how much of ParaView can be driven from the web. Table 2 lists it.

Feature	What it does	Notes
Source	Wavelet, or a .vti file from the Showcase A datasets	Only foot_64 was tested as a file
Filter	Contour, Slice, Clip, Threshold, Streamlines, Glyph	All six render in local mode too
Colour by,	Colours the result by an	The scalar bar is drawn by the server,

colour map, range	array, with a chosen colour map	so it shows in remote mode only
Volume rendering	Server-side volume rendering	Remote mode only
Screenshot, save and load state	Saves an image and the app's settings	Load restores the app's own settings file, not the Paraview .pvsm file
Multiple clients	Two browsers share one session	They share state and camera. A page opened later starts from the server's current state

Table 2

The wavelet has no vector field, so Streamlines and Glyph run on the gradient of the wavelet's scalar. And the Contour filter outputs only the contoured array, so colouring it needed an Elevation filter added first. The clients connected counter only ever goes up, so it is labelled "so far”

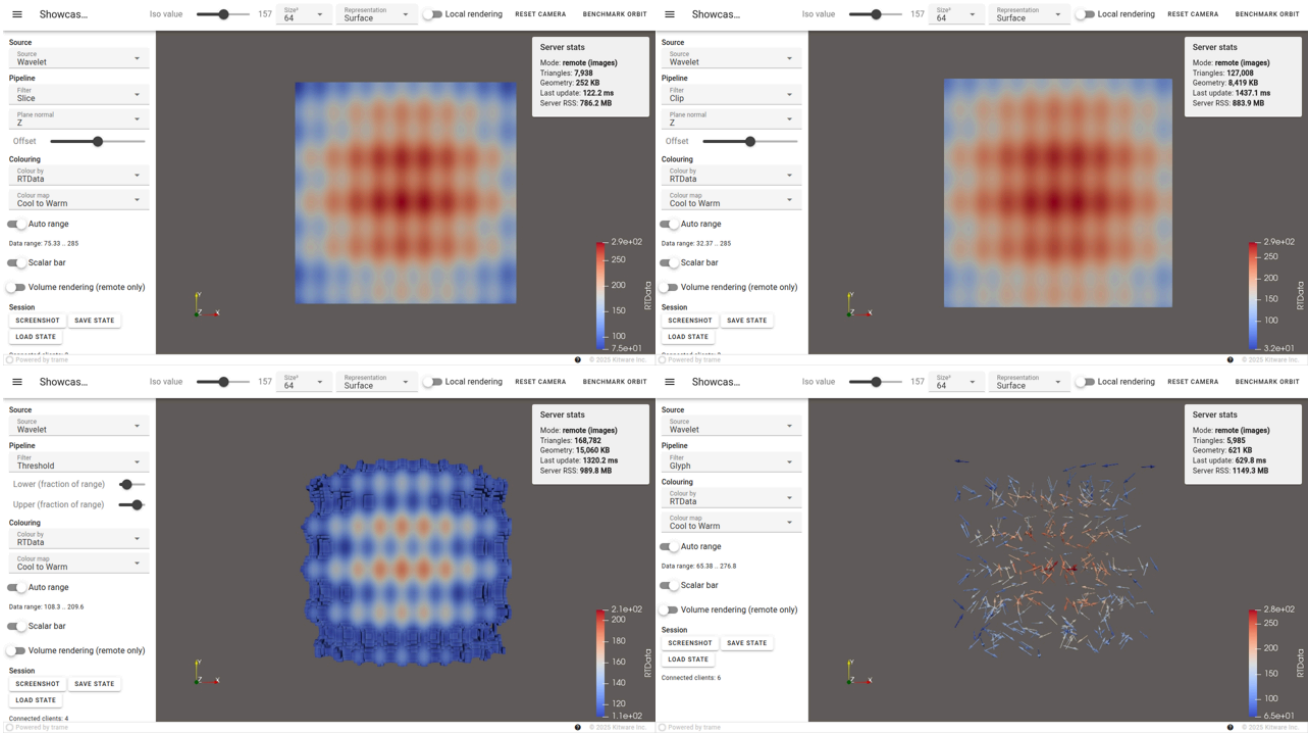


Fig6: Four pipelines selectable in Showcase B: Slice, Clip, Threshold and Glyph, coloured by RTData (64³ wavelet, remote mode)

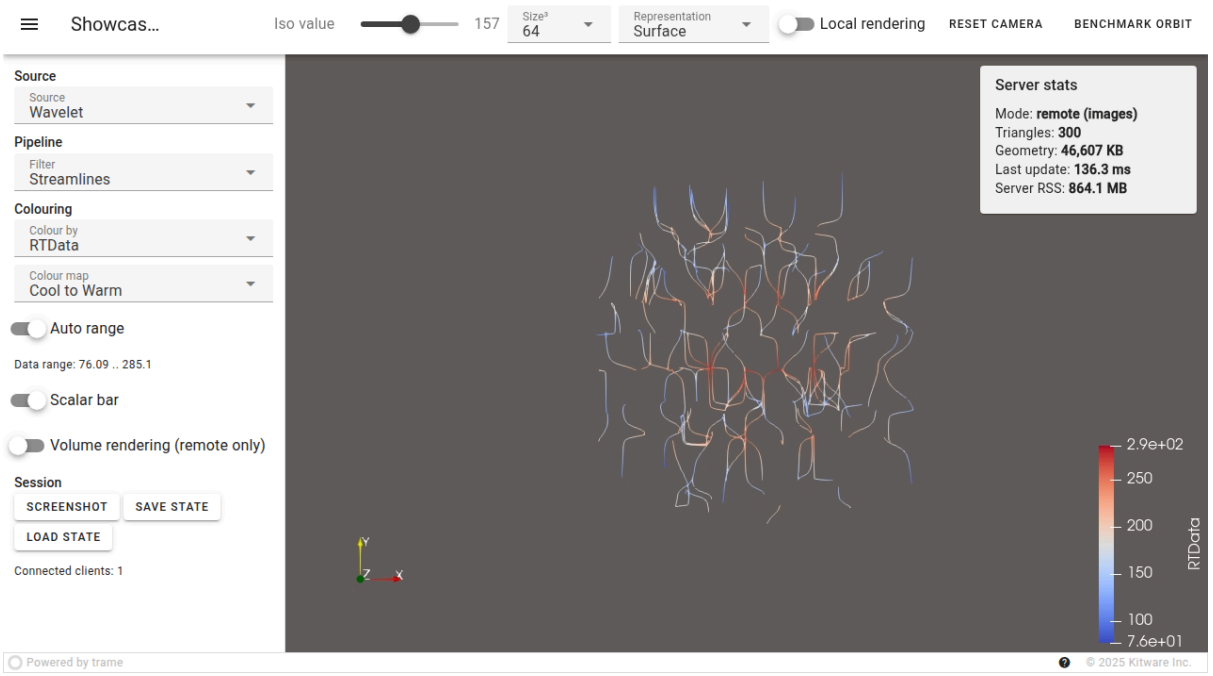


Fig 7:depicting stream lines

The Local rendering switch is the main feature of this showcase. With it off, the server renders and streams images. With it on, the server sends the geometry once and vtk.js draws it in the browser. The geometry is 3.0 MB at 64³ (66,260 triangles), 12.3 MB at 128³ (272,634 triangles) and 49.6 MB at 256³ (about 1.1 million triangles)

Topology analysis with TTK

A separate app,uses the Topology Tool Kit to find the critical points of a scalar field and draw its persistence diagram, which vtk.js cannot do in the browser. TTK 1.3.0 comes bundled with ParaView 6.2.0 as a plugin, so nothing had to be installed, and it loads in about 6.6 seconds. The app takes the critical points from the persistence pairs and shows minima, saddles and maxima as coloured spheres, with a slider that filters out features below a chosen persistence. I tested it on a field made from five Gaussians, where the answer is known and the app found exactly five maxima at the 5% and 10% thresholds. TTK crashes pvpython if the scalar field is not named explicitly, and the app always names it. The app works in remote mode only.

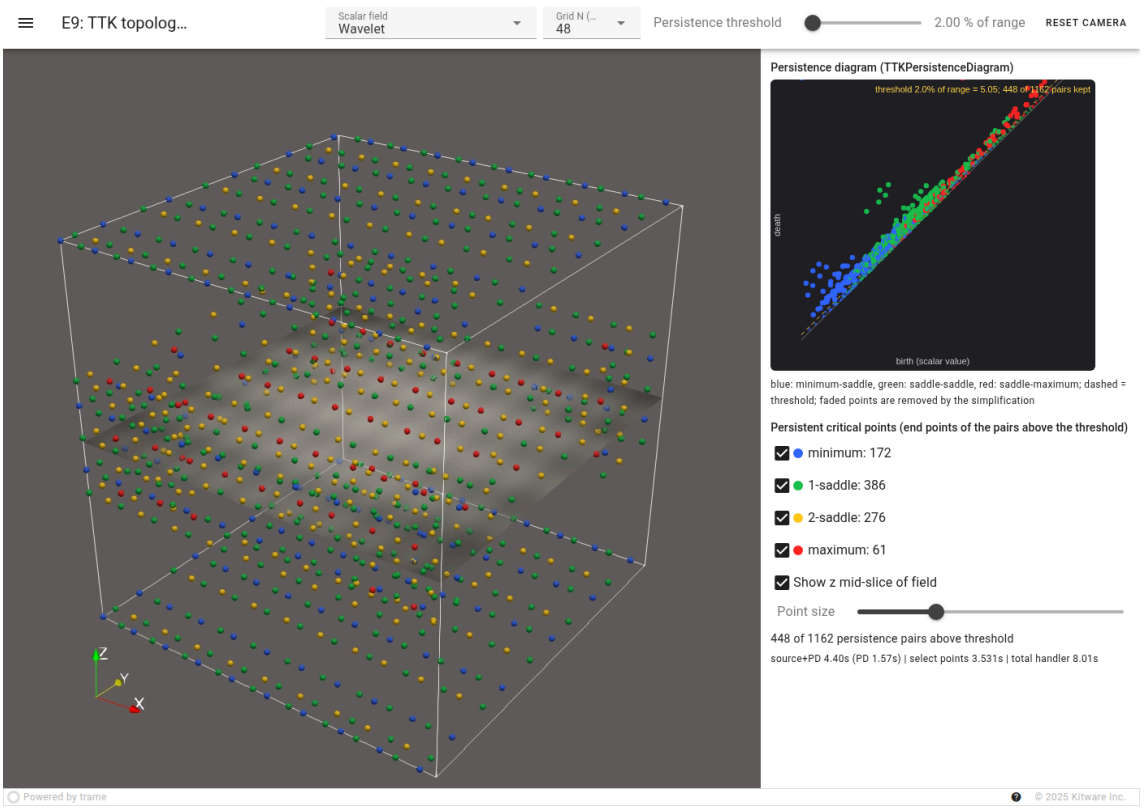


Fig8: TTK critical points for a sum of five Gaussians (48³ grid), persistence threshold 5%. Exactly five maxima (red) remain, matching the known answer

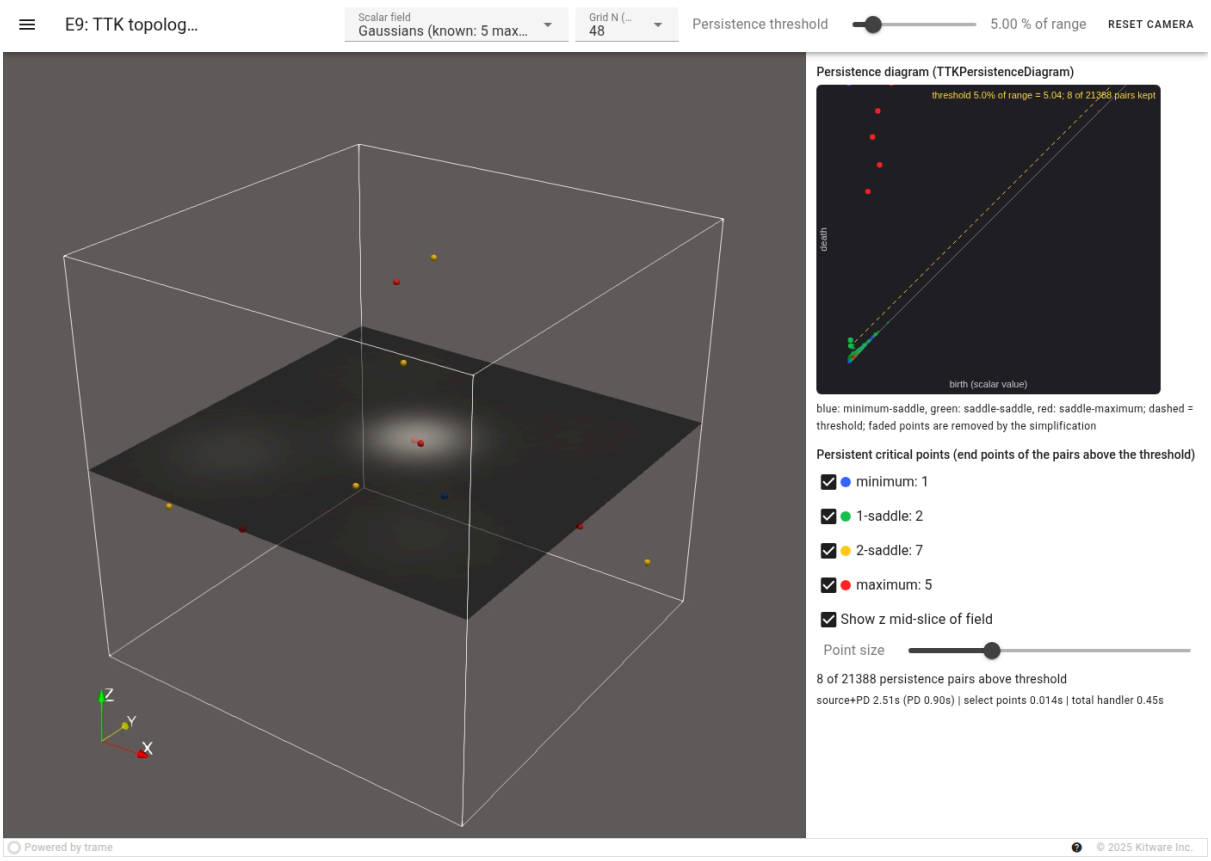


Fig9: The same analysis for the Wavelet field at a 2% threshold. Many small features survive, because the field oscillates throughout the volume

My observations:

Switching between local and remote: The switch makes the difference between the two approaches easy to see. In remote mode the picture is a stream of images from the server, and the camera moves only as fast as the round trip allows. In local mode the server sends the geometry once, and after that the browser rotates it without talking to the server at all The local mode also felt much faster to rotate than the remote mode

Sharing a session: When I opened a second browser on the same server, both showed the same scene and moved together, and a page opened later started from the server's current state. This is useful for teaching or demos. The "clients connected" counter only goes up and never drops when a page closes, which is why the label says "so far"

Memory: The server's memory use kept climbing as I switched filters from about 750 MB at the start to nearly 1,700 MB after ten or so changes, and it did not fall back when I returned to the first filter.

Topology: The TTK app was the clearest example of something Showcase B can do that Showcase A cannot. Changing the source took about a second and a half, and moving the threshold slider was nearly instant. On the five-Gaussian field it found exactly the five maxima I expected

6.Feature comparison matrix

To compare what the three systems can do, and not only how fast they are, I built a matrix of 75 capabilities in seven groups: rendering, interaction, data, architecture, collaboration and deployment, development, and server-side processing. The columns are vtk.js 37.4.6, ParaView 6.2.0 with trame 4.0.0, and the legacy ParaViewWeb applications. Each cell holds one of four values. Demonstrated means I built or ran it and kept a screenshot or log. Documented means an official source or the installed code says it exists, and I checked that. Not supported means I looked and it is absent. Unknown means I could not verify it.

	Demonstrated	Documented	Not supported	Unknown
vtk.js	23	29	19	4
ParaView + trame	27	38	4	6
Legacy ParaViewWeb	0	33	1	41

The legacy column has no demonstrated cells because I did not build or run those applications. Their official pages list no features, so most cells there are Unknown.

Selected differences:(rows from matrix with numbers indicated in bracket)

Capability	vtk.js	ParaView + trame
Volume rendering (1)	Demonstrated	Demonstrated, server-side
Picking and probing (22)	Demonstrated	Demonstrated, on the server
Picking and probing (22)	Demonstrated	Unknown (not verified
Distance measurement (23)	Demonstrated	Documented
Filter catalogue (65)	About 50 filters, documented	288 filters, documented
Topology analysis (71)	Not supported	Demonstrated(TTK)
Unstructured grids and AMR (41)	Not supported	Documented
Shared sessions (52)	Not supported	Demonstrated
Hybrid local/remote rendering (45)	Not supported	Demonstrated
Static hosting, no server (57)	Demonstrated	Not supported

WebXR (48)	Documented, emulator test only	Documented
WebGPU (47)	Documented; my probe gave no render	Not supported in trame-vtk 2.11.17

Limits of this comparison: A "Demonstrated" cell means it worked in my showcase on one machine, not that the feature is complete or fast. A "Documented" cell means a source says it exists, not that I ran it. WebXR was tested only in an emulator, WebGPU produced no render, and Jupyter was not demonstrated.

7.Benchmarking

Three conditions were measured:
1) the vtk.js volume viewer (A-local),
2) the trame app with the geometry drawn in the browser (B-local)
3) trame app drawn on the server (B-remote).
Each was run at three sizes, 64³, 128³ and 256³, with three repetitions and a fresh browser start every time. All runs were on one laptop: an Intel i7 with 31 GB of RAM and an integrated Intel Iris Xe GPU, running PopOS with Chrome 151 and a 60 Hz display.

Results:

Condition	Size	First render (ms)	FPS median	FPS slowest second	Browser memory (MB)	Server memory (MB)	WebSocket bytes in the orbit
A-local	64	216	59.9	59.5	19.6	n/a	n/a
A-local	128	107	59.9	59.5	21.3	n/a	n/a
A-local	256	435	30.0	25.6	41.0	n/a	n/a
B-local	64	376	60.0	60.0	10.4	709	0
B-local	128	357	60.0	60.0	11.7	757	0
B-local	256	356	60.0	60.0	11.7	958	0
B-remote	64	400	20.0	20.0	10.5	732	3.9 MB
B-remote	128	366	19.0	19.0	9.9	778	4.0 MB
B-remote	256	374	21.0	20.0	11.2	980	4.0 MB

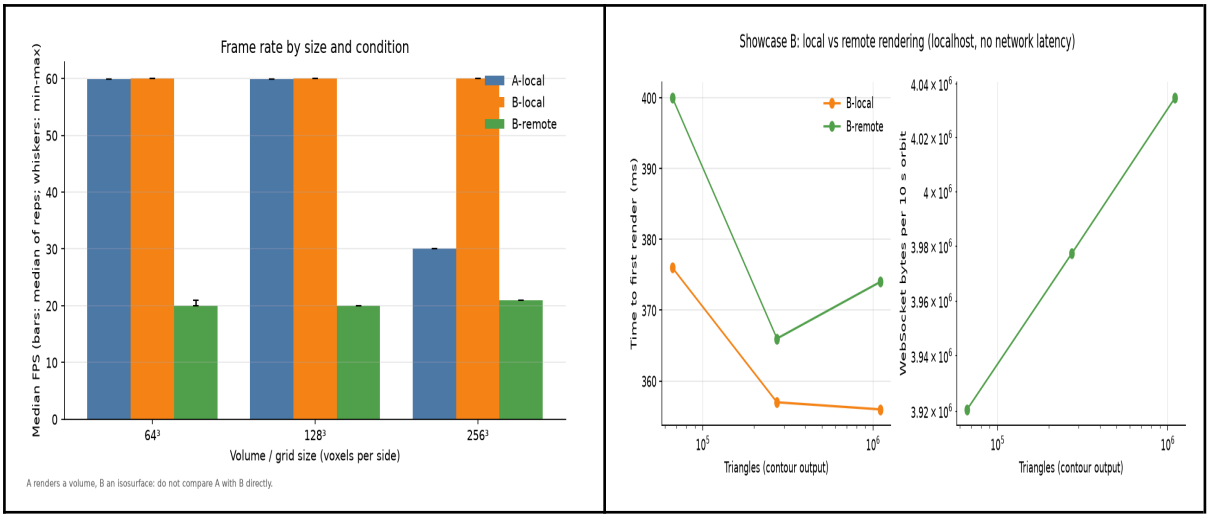


Fig 10: Frame rate by size and local vs remote FPS rendering speed graphs

The geometry in Showcase B was 66,260 triangles (3.0 MB) at 64^3, 272,634 triangles (12.3 MB) at 128^3 and 1,103,026 triangles (49.6 MB) at 256^3. In local mode that geometry is sent once, when the mode or size changes, and not at all during the orbit.

Discussions:

Local rendering held the display limit. B-local ran at 60 fps at all three sizes and sent no data during the orbit, so once the geometry has arrived the browser needs nothing more from the server. For these scenes, up to a million triangles, the integrated GPU was not the limit.

The remote figures show the input limit, not the render speed. B-remote stayed at about 20 fps at every size, and the traffic stayed at about 4 MB per 10 seconds. Because the figure did not change when the scene grew by a factor of 17, it was not limited by rendering. The cause is trame's mouse handler, which sends at most one event every 33 ms, so a scripted drag cannot go much above 20 to 30 events per second. What the remote numbers therefore tell us is how fast input can be sent. They do not tell us how fast the server can render

Memory: The browser used between 10 and 41 MB of JavaScript memory, rising with size only in Showcase A. In Showcase B the server's memory grew with size, from 709 to 980 MB, and that is a cost only the server-side design pays.

The trade-off: Local mode costs a single transfer (3 to 50 MB here) and then nothing. Remote mode costs a steady stream of about 400 kB per second but needs no transfer up front. On localhost both are cheap

Limits of the benchmark:Localhost has almost no network delay, which flatters remote rendering. Showcase A renders a volume and Showcase B an isosurface, so their frame rates are not comparable. Frame rates are capped by the 60 Hz display, so a value of 60 cannot separate two conditions. There were only three repetitions per cell, and the first render in remote mode sometimes took 2.7 to 2.9 seconds in the third repetition for reasons I did not find, which the median hides

8. ISO/IEC 25010 evaluation

Method:I scored each showcase on the eight quality characteristics of ISO/IEC 25010 on a scale of 1 to 5, using the evidence from the installation log, the feature matrix and the benchmark. The scores are my own judgement and are not from a user study

Scores:

Characteristic	Showcase A (vtk.js)	Showcase B (trame + ParaView)
Functional suitability	4	4
Performance efficiency	3	3
Compatibility	3	2
Usability	4	3
Reliability	4	3
Security	5	2
Maintainability	4	3
Portability	4	3
Total (of 40)	31	23

Justifications:

Functional suitability (A 4, B 4). A renders volumes with three presets and a Tools panel of ten features, but it has no server-side processing, and WebGPU and mesh formats are only partly covered. B offers six filters, volume rendering, state saving, shared sessions and topology analysis, but several features exist in one mode only.

Performance efficiency (A 3, B 3). A held 60 fps up to 128^3 and fell to a median of 30 fps at 256^3 on the integrated GPU. B's local mode held 60 fps at all sizes, but remote mode showed about 20 fps, limited by mouse input, and the server used 700 to 980 MB, growing during use.

Compatibility (A 3, B 2). A needed an extra package alias for Vite, and I only verified Chrome, not Firefox. B had the most compatibility trouble: the Python version must match ParaView's, a pip-installed vtk conflicts with ParaView's own, and trame-vtk had no serializer for ParaView 6.2's data model.

Security (A 5, B 2). A is static files only, with no server code to attack. B runs a WebSocket server with Python on the server side, and I added no authentication or TLS. That was a deliberate non-goal, but it limits real deployment.

Portability (A 4, B 3). A runs in any WebGL2 browser, though I only tested Chrome on Linux. B needs a 901 MB ParaView install and a Python server, and the Windows scripts were never run.

Reliability (A 4, B 3). A was stable once it ran. B failed silently in local mode, crashes pvpython if TTK's scalar field is not named, and showed memory growth and variable timing.

Usability (A 4, B 3): A's panel is simple, though the Tools panel is long. B's drawer works, but switching to local mode quietly removes the scalar bar and volume rendering, and the clients counter only goes up.

9. Munzner nested-model analysis

Approach: I analysed each showcase with the four levels of Munzner's nested model. One limit applies at the top level: the domain situation is assumed, not studied. I did not interview real users, so it describes who the tools are for in principle.

The four levels:

Level	Showcase A (vtk.js)	Showcase B (trame + ParaView)
Domain situation	A reader of a web article or a student who wants to look inside 3D scans (foot, aneurism,bonsai, tooth) without installing software	An analyst or teacher who needs ParaView's processing but wants to use it from a browser, on data that may be too large to ship to the client
Data and task abstraction	Data: a 3D scalar volume (8-bit values on a regular grid). Tasks: look at the overall structure, pick out one material such as bone or vessels, read a value at a point, measure a distance, look at a plane	Data: a 3D scalar field (the wavelet or a dataset), plus fields derived on the server (a gradient vector field, persistence pairs). Tasks: extract surfaces at a value, cut and clip, select by range, follow flow, colour by an array, find and filter topological features
Visual encoding and interaction idiom	Direct volume rendering, where a transfer function maps value to colour and opacity,isosurface; three orthogonal slices; a scalar bar. Interaction: drag to rotate, zoom, sliders, click to probe, clipping planes, a transfer-function editor, a line widget	Surfaces coloured by a colour map, slices, glyphs, streamlines, spheres for critical points, and a scatter plot for the persistence diagram. Interaction: a filter drawer, sliders for iso value and threshold, mouse camera, a local/remote switch
Algorithm	GPU ray casting in a WebGL2 shader for the volume; marching cubes in JavaScript on the browser's main thread for the isosurface; picking by ray casting	Marching cubes and other filters in ParaView on the server; compression and streaming of rendered images over a WebSocket (remote) or serialising geometry for vtk.js (local); TTK's persistence computation for topology

How the levels constrain each other:
1)Task to encoding.:Probing a value, measuring a distance and comparing materials need an encoding where values can be read. In A the transfer function hides everything outside the chosen range, which suits "show me the bone" but makes "what is the value here" depend on the probe tool.
2) Encoding to algorithm. In A, volume rendering is practical in the browser only because ray casting runs on the GPU. At 256^3 the integrated GPU became the limit (about 30 fps), so the algorithm level set the ceiling for the idiom above it. The browser isosurface runs on the main thread, so it limits usable data size.
3)Data size and the rendering location. In B, the choice between sending images and sending geometry follows from the data. Geometry for a 256^3 isosurface is about 50 MB, which is fine to send once, but data with far more triangles would make local mode impractical. That is where the server-side design pays off

4)Domain to architecture. The requirement "no installation" pushes toward A. The requirement "heavy processing, big data" pushes toward B.

Threats at each level:

Domain: no real users were consulted, so the needs described here are assumed

Abstraction: the tasks listed are the ones I could demonstrate, not a survey of what users want

Idiom: I have not tested whether the transfer-function presets or the controls are understandable to a new user.

Algorithm: performance was measured on one integrated GPU, so the 256^3 ceiling would differ elsewhere.

10. Advantages and Disadvantages

1)vtk.js

Advantages:

1) **No server and no installation for the viewer**: Showcase A is static files, so it can be hosted anywhere, including GitHub Pages, and readers just open a page. (refer Sections 2 and 4)
2)**Smooth interaction once the data has loaded**: Rotating and zooming never go over the network. A-local held 60 fps up to 128^3. (refer Section 7.)
3) **A broad toolbox in the browser:** I demonstrated an isosurface, orthogonal slices, probing, clipping, a transfer-function editor, a measurement widget, mesh loading and image export. (ref.Sections 4 and 6)
4) **Small and easy to keep running:** The memory use was small, versions are pinned, and the code is plain JavaScript. (Sections 3 and 8.)
5)L**ow security risk**: There is no server code to attack. (Section 8.)

Disadvantages:

1) **Limited by the user's hardware.** A 256^3 volume dropped to about 30 fps on an integrated GPU, and the whole volume must fit in browser memory. (Section 7.)
2)**No server-side processing.** There is no topology analysis, no large filter catalogue, and no support for unstructured grids or AMR data. (Section 6.)
3)**Some features are heavy or unfinished.** The isosurface runs on the main thread, WebGPU gave no render in my test, and WebXR was tested only in an emulator. (Section 4.)
4)**Cheap interaction in local mode.** After the geometry arrives, local mode held 60 fps at all sizes and sent nothing. (Section 7.)

2) ParaView with trame (server-side and hybrid rendering)

Advantages:

1) **The full power of ParaView from a browser.** There are 288 filters, Python scripting, server-side volume rendering and plugins such as TTK, which gave topology analysis that vtk.js cannot do. (Sections 5 and 6.)
2)**Large data stays on the server**. Only images or geometry travel, so the client needs little memory. In remote mode the browser used about 10 MB. (Section 7.)
3)**A hybrid option**. One scene can be drawn on the server or in the browser, and shared sessions let several people look at the same state. (Section 5.)
4) **Cheap interaction in local mode**. After the geometry arrives, local mode held 60 fps at all sizes and sent nothing. (Section 7.)

Disadvantages:

1) **Heavy to install and run**. ParaView is a 901 MB download, the Python environment must match it, and it needs a running server. (Section 3.)
2)**Fragile at the edges**. Local mode failed silently with ParaView 6.2 because trame-vtk lacked a serializer, TTK crashes without a named field, and the server's memory grew while I switched filters. (Sections 5 and 8.)
3)F**eatures differ between modes**. The scalar bar and volume rendering work in remote mode only. (Section 5.)
4)**Remote interaction depends on the network and the browser.** The mouse handler limits input to about 30 events per second, and the orbit timing varied with browser state. (Section 7.)
5) **Security needs extra work**. The WebSocket server has no authentication or TLS in my setup. (Section 8.)

Which to pick?

If you need	Better fit
A page anyone can open, with no setup	vtk.js
Smooth interaction for modest data	vtk.js, or trame in local mode
Heavy processing, large data, or ParaView's filters and plugins	ParaView with trame
several people looking at one session	ParaView with trame
The least maintenance effort	vtk.js

11.Legacy ParaViewWeb applications

I did not build or run the legacy applications, so this section is background from documentation and release notes. It is not an evaluation of them.

What they were:ParaViewWeb began as a JavaScript framework for building interactive visualization pages on top of VTK or ParaView. Around it Kitware built several applications. Visualizer was a web version of the ParaView desktop client, launched with :
pvpython -m paraview.apps.visualizer.
ParaView Lite was a simpler, more focused interface for controlling processing and visualization remotely. Divvy was an application for exploring relationships between variables. LightViz was a tailored interface for scientific visualization

What happened to them: The paraviewweb repository states that it is in maintenance mode, with no new features, and recommends trame for new 3D web applications. In 2024 a Kitware developer proposed removing Visualizer, Lite and Divvy from the ParaView bundle, citing easier maintenance of the binary releases and freedom from a fixed wslink version. The ParaView 5.13.0 release notes then state that they "are not bundled with ParaView anymore". They live in separate repositories and can still be installed into a Python virtual environment. LightViz's own README says it is deprecated and no longer supported. In my installed ParaView 6.2.0, the paraview/apps folder contains only a trame launcher, a Glance stub that reports it is unsupported, and a packages helper, with no Visualizer, Lite or Divvy modules.

What replaces them: I found no source that names a drop-in replacement for the Visualizer, Lite or Divvy interfaces. Kitware's direction is trame together with vtk.js, which is what the two showcases use. A trame application has to be written, so the ready-made general interface that Visualizer provided has no direct counterpart.

12. Research gaps and future work

Gaps I found:

1)**Keeping up with ParaView's data model.**
trame-vtk 2.11.17 had no serializer for ParaView 6.2's data collection, and local mode failed silently. Better automatic checks, or errors for unsupported types, would remove a whole class of proble

2)**Feature parity between the two modes.**
Switching to local mode removed the scalar bar and volume rendering. A hybrid that offers the same features in both modes would be more useful.

3)**Help in choosing local or remote.**
My benchmark shows the trade-off between a one-time geometry push and a steady image stream, but nothing helps a user decide for their own data size and network. A simple model or tool that predicts the better choice would fill this gap.

4)**Measuring remote rendering fairly**.
The frame rate in my scripted runs was limited by trame's input throttle, and the server-side orbit timing changed with the state of the browser. A standard way of benchmarking web-based remote rendering that avoids these pitfalls would make results comparable.

5) **Topology in the browser.**
Topology analysis worked only on the server with TTK. There is nothing like it in vtk.js, so browser-only users cannot use it.

6) **Documentation.** Kitware itself calls some of the legacy ParaViewWeb documentation outdated for newcomers, and I found no single guide that compares vtk.js, trame and the older tools.

Things I did not test:

1)**Real networks.**
All runs were on localhost, which flatters remote rendering. Throttled or real wide-area connections would show where each approach breaks.

2)**Larger data:**
The biggest volume was 256^3. Multi-gigabyte volumes, unstructured grids and time series would stress both designs differently.

3) **More hardware**
I used one laptop with an integrated GPU on Linux. A discrete GPU, other browsers (Firefox was not verified) and Windows (the scripts were never run) could change the picture.

4)**Users.**

I did not run a usability study, so the usability scores in Section 8 reflect only my own use.

5)**Scaling to many users**.
Shared sessions worked for two browsers. Per-user isolation, authentication and load were not tested.

6) **WebGPU and WebAssembly.**
vtk.js's WebGPU support is still catching up with WebGL (an open parity issue), and trame has a VTK.wasm option, trame-vtklocal. I did not evaluate either.

13.Conclusion and recommendations

This report evaluated vtk.js for client-side rendering and ParaView 6.2 with trame for server-side and hybrid rendering, using two showcase applications, a feature comparison, a benchmark and an ISO/IEC 25010 scoring. I return here to the four research questions.

1)**What can the two systems do?**
vtk.js proved capable for a library that runs entirely in the browser. In my viewer it handled volume rendering, isosurfaces, slices, probing, clipping and measurement. ParaView with trame went further on the server: it offered a catalogue of 288 filters, server-side volume rendering and plugins, and with TTK it did topological analysis that vtk.js cannot do. Of 75 capabilities I compared, I demonstrated 23 for vtk.js and 27 for ParaView with trame.

2)**How do browser and server rendering compare?**
On localhost and an integrated GPU, rendering in the browser held the display limit (60 fps) for everything I tried except a 256^3 volume in vtk.js, which fell to about 30 fps. In local mode no data moved after the first push of 3 to 50 MB. Remote mode streamed about 4 MB per 10 seconds at every size, and its frame rate showed the limit of trame's input handling and not the cost of rendering. So my data cannot say which mode renders faster in general. It shows that the difference is mostly about data transfer. A real network and larger data would be needed to say more.

3) **How easy are they to install, and how sustainable?**
The installation worked, but not without effort. vtk.js needed one fix, while the ParaView side needed a 901 MB download, matching Python versions and a workaround for a missing serializer. A scripted clean-room test now passes, but I tested Linux only. On sustainability, ParaViewWeb is in maintenance mode, and its applications were removed from the ParaView bundle in 5.13, so new work should be built on trame and vtk.js.

4) **When is each approach suited, and where does it break down?**
vtk.js fits sharing a visualization as a web page with no installation and no server, as long as the data fits in the browser and the GPU can draw it. ParaView with trame fits heavy processing, large data and shared sessions, at the cost of a server, a heavier install and some fragility at the edges. The hybrid mode lets one scene use both, but it does not give the same features in each mode.

Recommendations
1. For a simple article or teaching page with modest data, use vtk.js.
2. For heavy processing, large data or ParaView's filters and plugins, use ParaView with trame, and budget time for setup.
3. For a new project, don't start from the legacy ParaViewWeb applications.
4. Before relying on local mode, check that your ParaView and trame-vtk versions support your data type, because failures can be silent.

CONCLUSION: choose vtk.js when the aim is simple sharing with no setup, and ParaView with trame when the aim is processing power and large data. For new projects, build on

these two and not on the legacy ParaViewWeb applications, which are in maintenance mode.

Appendix

This appendix gathers the supporting material that the report refers to. The files named below are in the project repository.

A. Software versions:

Component	Version	Notes
Operating system	Pop!_OS (Linux 7.0.11, x86_64), native	Ubuntu-based; not WSL
CPU / RAM	Intel Core i7-1185G7, 8 logical cores, 31 GB	
GPU	Intel Iris Xe (TigerLake-LP GT2)	Integrated; no discrete GPU
Browser	Google Chrome 151	WebGL and WebGPU hardware accelerated
Node.js / npm	24.13.0 / 11.6.2	Design target was 22 LTS
Vite	8.3.5	Pinned exactly
@kitware/vtk.js	37.4.6	Pinned exactly
events (npm)	3.3.0	Polyfill needed by vtk.js in Vite
Python	3.12.3 (system)	
ParaView	6.2.0 (MPI Linux tarball)	Bundled Python 3.12.14
trame / trame-vtk / trame-vuetify	4.0.0 / 2.11.17 / 3.3.0	
trame-client / trame-server / wslink	4.1.4 / 4.1.0 / 2.6.0	
psutil	7.2.2	
vtk (pip, tools environment only)	9.7.1	Never installed in the ParaView environment
numpy / pandas / matplotlib (tools only)	2.5.4 / 3.1.0 / 3.11.2	

B. System and browser graphics information:

All results were produced on one laptop (source: bench/results/sysinfo.json and chromeGpuInfo.txt). The Chrome GPU report lists Canvas, rasterization, WebGL, WebGPU and WebGPU interop as hardware accelerated, and Vulkan as disabled. The WebGL renderer is ANGLE (Intel, Mesa Intel(R) Iris(R) Xe Graphics (TGL GT2), OpenGL ES 3.2) on Mesa 25.2.8, using the ANGLE OpenGL backend. The display refresh rate is 60 Hz, which caps the frame rates in the benchmark.

C. Installation log (selected entries):

Step	Duration	Problem	Resolution
Download ParaView 6.2.0 (901 MB)	4 min 10 s	None	Background download, about 3.6 MB/s
pip install trame packages in the ParaView environment	10 s	None	Pinned versions installed without conflict
npm install vtk.js and Vite	11 s	None	0 vulnerabilities reported
Dataset download from klacansky.com	-	HTTPS refused on port 443	Plain HTTP; SHA-256 checksums recorded

Step	Duration	Problem	Resolution
Test volume headsq.vti	-	URL returns vtk.js JSON directory format, not XML	Replaced by a generated wavelet volume
Showcase A first load in Vite	-	Blank page: Class extends value undefined	Installed events 3.3.0 and aliased it in the Vite config
Showcase B local mode	About 40 min	Empty scene, no error message	trame-vtk 2.11.17 has no serializer for vtkPartitionedDataSetCollection; one registered in app.py
Clean-room test, manual (fresh clone)	2 min 10 s	None	All 9 setup steps passed
Dataset host slow on the student's machine	Stalled	About 5 kB/s; old downloader had no stall detection	Resumable downloader: HTTP range, abort slow connections, retry
Scripted clean-room test	34 s	False pass caused by a leftover server on port 5173	Port check and cleanup fixed; 8 of 8 checks pass
Showcase B benchmark (18 runs)	About 25 min	None	Rows appended to results.csv
Showcase A in Firefox	-	Headless screenshot blank, inconclusive	Not verified

The full log, including every command and date, is report/install_log.md.

D. Raw benchmark data (all 27 runs):

Condition	Size	Rep	First render (ms)	FPS median	FPS slowest second	Browser memory (MB)	Server memory (MB)	WebSocket bytes
A-local	64	1	216	59.9	59.5	19.6	n/a	n/a
A-local	64	2	534	59.9	59.5	21.7	n/a	n/a
A-local	64	3	61	59.9	59.5	19.6	n/a	n/a
A-local	128	1	133	59.9	59.5	21.0	n/a	n/a
A-local	128	2	100	59.9	1.0	21.3	n/a	n/a
A-local	128	3	107	59.9	59.9	24.0	n/a	n/a
A-local	256	1	434	30.0	20.0	38.6	n/a	n/a
A-local	256	2	438	30.0	25.6	51.1	n/a	n/a
A-local	256	3	422	30.0	29.9	41.0	n/a	n/a
B-local	64	1	377	60.0	60.0	10.3	704	0
B-local	64	2	376	60.0	60.0	11.0	709	0
B-local	64	3	364	60.0	60.0	10.4	713	0
B-local	128	1	357	60.0	60.0	11.6	751	0
B-local	128	2	349	60.0	60.0	11.7	757	0
B-local	128	3	363	60.0	60.0	12.6	764	0
B-local	256	1	356	60.0	60.0	11.7	942	0
B-local	256	2	355	60.0	60.0	11.8	958	0
B-local	256	3	371	60.0	60.0	10.7	973	0

Condition	Size	Rep	First render (ms)	FPS median	FPS slowest second	Browser memory (MB)	Server memory (MB)	WebSocket bytes
B-remote	64	1	376	20.0	20.0	12.0	704	3.9 MB
B-remote	64	2	400	21.0	19.0	9.4	732	4.0 MB
B-remote	64	3	2720	20.0	20.0	10.5	751	3.9 MB
B-remote	128	1	361	20.0	19.0	11.4	751	3.9 MB
B-remote	128	2	366	20.0	20.0	9.4	778	4.0 MB
B-remote	128	3	2868	20.0	19.0	9.9	816	4.0 MB
B-remote	256	1	374	21.0	20.0	12.0	942	4.0 MB
B-remote	256	2	350	21.0	20.0	11.2	980	4.1 MB
B-remote	256	3	2723	21.0	20.0	10.8	1028	4.0 MB

Size is the edge length of the volume. WebSocket bytes are for the 10-second scripted orbit. In B-remote the third repetition at each size had a first-render time of 2.7 to 2.9 seconds, against about 0.37 seconds for the other two; the cause was not found. The file is bench/results/results.csv.

E. Feature matrix (status of all 75 capabilities):

Demonstrated means the feature was built and shown in this project. Documented means a source I checked says it is supported. Not supported means I looked and it is absent. Unknown means I could not verify it. The evidence behind each cell (file, figure or source) is in report/feature_matrix.md.

Capability	vtk.js 37.4.6	ParaView 6.2.0 + trame	Legacy ParaViewWeb
Rendering			
1. Volume rendering	Demonstrated	Demonstrated	Unknown
2. Isosurface / contour	Demonstrated	Demonstrated	Unknown
3. Orthogonal slice / MPR	Demonstrated	Demonstrated	Unknown
4. Arbitrary-plane cutting of geometry	Documented	Demonstrated	Unknown
5. Polygonal mesh / surface rendering	Demonstrated	Demonstrated	Documented
6. Threshold / extract by value	Documented	Demonstrated	Unknown
7. Glyphs	Documented	Demonstrated	Unknown
8. Streamlines / stream tracing	Documented	Demonstrated	Unknown
9. Point cloud rendering	Documented	Documented	Unknown
10. Tubes / ribbons	Documented	Documented	Unknown
11. Text / corner annotations	Documented	Documented	Unknown
12. Scalar bar / colour legend	Demonstrated	Demonstrated	Unknown
13. Axes / orientation widget	Demonstrated	Unknown	Unknown
14. Colour maps / lookup tables	Demonstrated	Demonstrated	Documented
15. Lighting / shading	Documented	Documented	Unknown
16. Clipping planes	Demonstrated	Demonstrated	Unknown
17. Volume cropping	Documented	Documented	Unknown
18. Multi-view / layouts	Documented	Documented	Unknown
19. Transparency (order-independent)	Documented	Unknown	Unknown
20. Ray tracing / path tracing	Not supported	Documented	Unknown
21. Charts / plots (line, histogram, parallel coords)	Not supported	Documented	Documented

Capability	vtk.js 37.4.6	ParaView 6.2.0 + trame	Legacy ParaViewWeb
Interaction			
22. Picking / probing a value	Demonstrated	Unknown	Documented
23. Distance / angle measurement	Demonstrated	Documented	Unknown
24. 3D interactive widgets	Documented	Documented	Documented
25. Camera controls (rotate/pan/zoom)	Demonstrated	Demonstrated	Documented
26. Alternative camera styles	Documented	Unknown	Unknown
27. Transfer-function editing (interactive)	Demonstrated	Documented	Documented
28. Animation / time-series playback	Documented	Documented	Documented
29. Screenshot / image export	Demonstrated	Demonstrated	Documented
30. Save / load pipeline state	Unknown	Demonstrated	Documented
Data			
31. Read VTK XML ImageData (.vti)	Demonstrated	Demonstrated	Documented
32. Read PolyData (.vtp, legacy .vtk)	Documented	Documented	Documented
33. Mesh formats (STL, OBJ, PLY, glTF, Draco)	Documented	Documented	Unknown
34. Scientific formats (Exodus, CGNS, NetCDF, HDF5, Xdmf, ADIOS2, EnSight)	Not supported	Documented	Documented
35. Image stacks / medical (TIFF, DICOM, NIfTI)	Unknown	Unknown	Unknown
36. Large data / out-of-core / LOD	Documented	Documented	Documented
37. Distributed (MPI) processing	Not supported	Documented	Unknown
38. In-situ (Catalyst)	Not supported	Documented	Unknown
39. Remote file browsing	Documented	Documented	Documented
40. Time-varying data	Documented	Documented	Documented
41. Unstructured grids / AMR	Not supported	Documented	Unknown
42. Data export / write	Documented	Documented	Documented
Architecture			
43. Client-only rendering (no server compute)	Demonstrated	Demonstrated	Documented
44. Server-side rendering (image stream)	Documented	Demonstrated	Documented
45. Hybrid (switch local/remote)	Not supported	Demonstrated	Documented
46. WebGL2	Demonstrated	Demonstrated	Unknown
47. WebGPU	Documented	Not supported	Unknown
48. WebXR / VR	Documented	Documented	Unknown
49. WebAssembly	Unknown	Documented	Unknown
50. Offscreen / headless server rendering	Not supported	Documented	Documented
Collaboration and deployment			
51. Multi-user sessions (isolated)	Not supported	Documented	Documented
52. Shared state across browsers (one session)	Not supported	Demonstrated	Unknown
53. Session launcher	Not supported	Documented	Documented
54. Jupyter embedding	Not supported	Documented	Unknown
55. Docker deployment	Not supported	Documented	Documented
56. Authentication / access control	Not supported	Documented	Documented
57. Static (no-server) hosting	Demonstrated	Not supported	Not supported
Development			
58. Primary language / API	Demonstrated	Demonstrated	Documented
59. TypeScript typings	Documented	Not supported	Unknown

Capability	vtk.js 37.4.6	ParaView 6.2.0 + trame	Legacy ParaViewWeb
60. Licence	Documented	Unknown	Documented
61. Maintenance status	Documented	Documented	Documented
62. Install effort (this project)	Demonstrated	Demonstrated	Unknown
63. Docs / examples	Unknown	Documented	Documented
64. Package-manager env for extensions	Demonstrated	Documented	Unknown
Server-side processing			
65. Catalogue of filters	Documented	Documented	Documented
66. Python scripting	Not supported	Demonstrated	Documented
67. Programmable Filter / Source	Not supported	Documented	Unknown
68. Calculator / derived arrays	Documented	Documented	Unknown
69. Plugins	Not supported	Demonstrated	Documented
70. Browser-side compute filters	Demonstrated	Not supported	Documented
Rows added later			
71. Topological analysis (persistence pairs, critical points)	Not supported	Demonstrated	Unknown
72. Persistence diagram display	Not supported	Demonstrated	Unknown
73. Mesh file loading in the page (.obj)	Demonstrated	Documented	Unknown
74. Three orthogonal slices shown together	Demonstrated	Documented	Unknown
75. Runtime-selectable pipeline filters (Contour, Slice, Clip, Threshold, Streamlines, Glyph)	Documented	Demonstrated	Unknown

F. Dataset credits and licences:

The volumes come from the Open Scientific Visualization Datasets collection by Pavol Klacansky (klacansky.com/open-scivis-datasets), which states the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 licence. Acknowledgements from the collection: foot and aneurism, volvis.org and Philips Research, Hamburg, Germany; bonsai, volvis.org and S. Roettger, VIS, University of Stuttgart; tooth, GE Aircraft Engines, Evendale, Ohio, USA. The wavelet volume is generated by the analytic Wavelet source of VTK and ParaView. The datasets are downloaded by the setup script and are not committed to the repository.

G. Reproducing the results:

The scripts are in the project repository (github.com/bala-skv/paraview-vtk.js). On Linux, from the repository root, with ParaView 6.2.0 installed:

```
export PVPYTHON=<path to ParaView 6.2.0 bin/pvpython>
scripts/setup.sh # environments, packages, datasets
scripts/run_a.sh # Showcase A at http://localhost:5173
scripts/run_b.sh # Showcase B at http://localhost:8080
scripts/cleanroom_test.sh # fresh clone, setup and smoke checks
bench/run_all_b.sh # Showcase B benchmark (GPU string required)
.venv/bin/python bench/plot_results.py # tables and figures
```

The PowerShell scripts for Windows were written but never run. The README in the repository describes each step.